

COMPUTATIONAL SOCIAL SCIENCE · TUTORIAL

Nostalgia on YouTube

Collecting and analysing platform data with the YouTube Data API, Google Colab and Hugging Face

Enes Abanoz · Faculty of Communication, Ondokuz Mayıs University

Department of Sociology and Social Research, University of Trento
September 22, 2026

What we will build today

1

Get API access

A free YouTube Data API key

2

Census a channel

596 videos, full statistics

3

Harvest comments

21,469 comments from one 1997 performance

4

Detect languages

lingua — the audience is not who you expect

5

Measure nostalgia

Dictionary vs. zero-shot transformer

The case

Andrea Bocelli Con Te Partirò

Performed **1997**

Uploaded **2015**

Comments **26,936**

Views **84 million**

Why this video: three clocks

An archival re-release carries three separate timestamps. Nostalgia is a claim about distance from a past — here we can measure that distance for every single comment.

1997

Performance

Piazza dei Cavalieri.
Fixed for every comment.

2015

Upload

Catalogue back-fill.
Fixed for every comment.

2015–26

Comment

This one varies —
and that is the variable.

A comment written in 2024 sits 27 years after the performance and 9 years after the upload. Most YouTube research collapses these into one date and loses the structure entirely.

01

The YouTube Data API

Getting a key, and understanding what it costs

One fact governs every design choice

A free key gives 10,000 quota units per day. The endpoints are not priced equally — and that asymmetry decides what research questions are cheap.

Endpoint	Units	Returns
search.list	100	≤50 results — same price for 1 or 50
videos.list	1	≤50 videos, full statistics
channels.list	1	≤50 channels
playlistItems.list	1	50 playlist entries
commentThreads.list	1	100 comments

10,000 units

= 100 searches

— or —

= 1,000,000 comments

Search is 100× the price of retrieval.

Our whole 596-video channel census cost under 100 units — because it never calls search.list. Framing a question around a known channel instead of a discovery problem changes the economics completely.

Create a Google Cloud project

- `console.cloud.google.com` — sign in with any Google account
 - Project dropdown in the top bar → New Project
 - Name it something recognisable (here: nextup-yt)
 - Wait for creation, then confirm it is the selected project
-
- Copy the key from the Credentials page
 - Click the key → API restrictions → Restrict key
 - Tick only YouTube Data API v3 → Save
 - A leaked restricted key can do nothing else

Open project picker (⌘ O)



You're working in nextup-yt

Project number: 912444268330  Project ID: nextup-yt 

[Dashboard](#) [Cloud Hub](#)

-  Create Gemini API key
-  Create an agent
-  Create a VM
-  Run a query in BigQuery
-  Deploy an application
-  Create a storage bucket

Quick access

-  APIs & Services
-  IAM & Admin
-  Billing
-  Compute Engine
-  Cloud Storage
-  BigQuery
-  VPC network
-  Kubernetes Engine

Join the Google Developer Program to learn new skills and dev tools at no cost.

Join today 

- Cloud Hub >
- Cloud overview >
- Solutions >
- Recently visited >

Favorite products

Favorite products appear here

Products

- Billing ☆
- IAM & Admin ☆ >
- Marketplace ☆
- APIs & Services** ☆ >
- Agent Platform ☆ >
- Compute Engine ☆ >
- Kubernetes Engine ☆ >
- Cloud Storage ☆ >

View all products

Get Agent Platform API key

- Enabled APIs & services
- Library
- Credentials
- OAuth consent screen
- Page usage agreements

Search (/) for resources, docs, products, and more

Search



Welcome

in nextup-yt

2444268330

Project ID: nextup-yt

Cloud Hub

API key

Create an agent

Create a VM

Run a query in BigQuery

Create a storage bucket

IAM & Admin

Billing

Compute Engine

BigQuery

VPC network

Kubernetes Engine

Join the Google Developer Program to learn new skills and dev tools at no cost.

Join today



API API Library

Search youtube data api v3

API Library > "youtube data api v3"

Vertex AI is now Gemini Enterprise Agent Platform. Read more in our [announcement blog](#).

Filter Type to filter

1 result

- Visibility ^
- Public (1)
- Category ^
- YouTube (1)



YouTube Data API v3
Google

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists,...

YouTube Data API v3

Product details



YouTube Data API v3

Google

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists,...

Enable Try this API

Click to enable this API

- Overview
- Documentation
- Support
- Related Products

Overview

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists, and channels.

About Google

Google's mission is to organize the world's information and make it universally accessible and useful. Through products and platforms like Search, Maps, Gmail, Android, Google Play, Chrome and YouTube, Google plays a meaningful role in the daily lives of billions of people.

Additional details

Type: [SaaS & APIs](#)
Last product update: 7/22/22
Category: [YouTube](#)
Service name: youtube.googleapis.com

- API APIs & Services
- Enabled APIs & services
- Library
- Credentials
- OAuth consent screen
- Page usage agreements

API/Service Details

Disable API

To call this API from your own applications, you may need to create credentials.

Create credentials



YouTube Data API v3

The YouTube Data API v3 is an API that provides access to YouTube data, such as videos, playlists, and channels.

By Google

Service name	Type	Status	Documentation	Explore	Support
youtube.googleapis.com	Public API	Enabled	Learn more	Try in API Explorer	Maintenance & Support

Metrics Quotas & System Limits Credentials

Select Graphs

4 Graphs

1 hour 6 hours 12 hours 1 day 2 days 4 days 7 days 14 days 30 days

Filters:

Versions

 v1 and v3

Credentials

 Unspecified, Anonymous...

Methods

 90 options selected

Traffic by response code

- API APIs & Services
- Enabled APIs & services
- Library
- Credentials
- OAuth consent screen
- Page usage agreements

Create credentials

1 Credential Type

Which API are you using?

Different APIs use different authentication platforms and some credentials can be restricted to only call certain APIs.

Select an API *

YouTube Data API v3

What data will you be accessing?

Different credentials are required to authorize access depending on the type of data that you request. [Learn more](#)

- ☐ User data 
Data belonging to a Google user, like their email address or age. User consent is required. This will create an OAuth client.
- ☒ Public data
Google data that is publicly available, like public Maps data showing restaurant information. This will create an API key.

Next

2 Your Credentials

Done

Cancel

- APIs & Services
- Enabled APIs & services
- Library
- Credentials
- OAuth consent screen
- Page usage agreements

Credentials + Create credentials Delete Restore deleted credentials

Create credentials to access your enabled APIs. [Learn more](#)

Remember to configure the OAuth consent screen with information about your application. [Configure consent screen](#)

API Keys

☐	Name	Bound account	Creation date	Restrictions	Actions
☐	 API key 1	—	Sep 8, 2026	—	Show key

OAuth 2.0 Client IDs

☐	Name	Creation date	Type	Client ID	Actions
No OAuth clients to display					

Service Accounts

[Manage service accounts](#)

☐	Email	Name	Actions
No service accounts to display			

Never type a key into a cell

Anything typed into a notebook cell is saved inside the .ipynb file — and that file is what you push to GitHub. The leak happens at collection time; you find out much later.

Colab Secrets

- Key icon in the left sidebar
- + Add new secret
- Name: YOUTUBE_API_KEY
- Toggle Notebook access on

```
from google.colab import userdata
API_KEY = userdata.get("YOUTUBE_API_KEY")

# The notebooks fall back gracefully:
#   Colab secret -> env var -> hidden prompt
# so the key never appears in the file.
```

02

Colab & repository structure

Everything runs from GitHub — nothing to install

One repository, three notebooks

```
bocelli-nostalgia/  
├── notebooks/  
│   ├── 01_channel_performance.ipynb  
│   ├── 02_harvest_and_prepare.ipynb  
│   └── 03_nostalgia_analysis.ipynb  
├── src/  
│   └── channel_client.py      API wrapper + quota meter  
├── data/  
│   ├── processed/           committed — notebooks load this  
│   └── raw/                  GITIGNORED — holds author IDs  
├── docs/  
│   ├── api_key_guide.md  
│   └── ethics_gdpr.md  
└── outputs/figures/
```

Every notebook starts the same way

```
if "google.colab" in sys.modules:  
    subprocess.run(["git", "clone", "-q",  
                    "https://github.com/eabanoz/"  
                    "bocelli-nostalgia.git"])  
    ROOT = Path("bocelli-nostalgia").resolve()  
os.chdir(ROOT)  
sys.path.insert(0, str(ROOT))
```

- No Google Drive mount, no manual upload
- Colab preloads pandas, torch, transformers, sklearn
- src/ is a real module, imported not pasted
- Data is committed — the tutorial runs without a key

data/raw/ is gitignored because it holds author_channel_id — personal data under GDPR. Pseudonymisation happens before anything leaves that folder.

What each notebook does

01

Channel performance

Full census of 596 videos. Quota accounting, the age confound, choosing the target video.

API key

no GPU

02

Harvest & prepare

21,469 comments. Pseudonymisation, cleaning, language detection, the three clocks.

optional

no GPU

03

Nostalgia analysis

Dictionary vs. zero-shot classification. Constructs by language and over time.

no key

GPU

Switches (HARVEST, SCORE) let notebooks 02 and 03 load the committed data instead of re-collecting or re-scoring.

03

Research steps

From a channel ID to a measured construct

Six decisions, not six commands

1 Select channel

Known channel beats a discovery problem — no search.list, no quota burn

2 Census uploads

596 videos. playlistItems, not search. Under 100 units

3 Choose one video

Comment count over view count — a nostalgia study needs text

4 Harvest comments

order="time", not relevance.
21,469 retrieved = 70.3%

5 Detect language

lingua, restricted set, confidence retained

6 Measure constructs

Dictionary AND classifier.
Compare them by language

Every one of these is a research decision that shapes the result — and each is reversible only by re-running.

What counts as nostalgia

Constructs from Svetlana Boym, *The Future of Nostalgia* (2001), and adjacent work. Restorative vs. reflective is the distinction that matters most: both are nostalgia, but only the first carries a claim about how things ought to be.

Personal memory

Autobiographical recall, usually family

Mortality / grief

The dead, funerals, memorials

Restorative

The past was better; a wish to return

Reflective

Bittersweet longing, without wanting return

Collective

Shared generational or national memory

Anemoia

Longing for a time never personally lived

These become hypotheses for a zero-shot classifier — the model has never seen a labelled example of any of them.

Now let's run it

`github.com/eabanoz/bocelli-nostalgia`

Open notebook 01 from the README badge. You do not need an API key to follow along — the data is in the repo. Get a key if you want to collect your own.